

Dilmurod Ibroximov

Data Engineer · Apache Airflow · ClickHouse · PostgreSQL · Python

I own the pipelines that carry Tashkent's city data into the warehouse and into reports. **38 DAGs** in Airflow pull **~12M rows** from **17 sources** every day. After the analytics moved to ClickHouse, reports build in **1.5 s** instead of 40.

now Data Engineer, Raqamli Rivojlanish Departamenti

city Tashkent, Uzbekistan

work office, hybrid or remote

transform_experience PostgresOperator queued —

Experience

Data Engineer

Raqamli Rivojlanish Departamenti

Digital Development Department, Tashkent

November 2025 — present **10 mo**

PIPELINES AND STORAGE

- Built 15 and maintain **38 DAGs** in Apache Airflow. Every day they collect **~12M rows** from 17 sources: agency APIs, district administration databases, XML and Excel exports.
- Set up retries, SLAs and failure alerts: **99.6%** of runs need no manual work, and the team sees a broken pipeline within 5 minutes.
- Designed PostgreSQL schemas for 9 domains. Partitioning and indexes made key queries **6× faster**.
- Moved analytical aggregates into ClickHouse (900M+ rows). On materialized views, reports build in **1.5 s** instead of 40.
- Built data-quality checks into every pipeline: completeness, duplicates, schema conformance. Broken records in the marts dropped **from 4% to 0.3%**.
- Replaced manual exports with Python and pandas scripts. Analysts save about **20 hours a week**.

INTEGRATIONS AND SERVICES

- Connected 11 external services over REST APIs; JSON and XML payloads are validated against schemas.
- Moved heavy work into Celery queues, up to 200k tasks a day. Redis caching cut the internal API response from 1.2 s to 150 ms.
- Stream data in real time over WebSocket to the operators' dashboard, under one second of latency.
- Added full-text search on Elasticsearch across 3M citizen requests, answering in under 200 ms.

INFRASTRUCTURE

- Packed 14 services into Docker Compose. A fresh environment comes up in 10 minutes instead of half a working day.
- Moved 60+ passwords and access keys out of code and .env files into HashiCorp Vault.
- Keep raw data and backups in AWS S3, and run separate ETL jobs on EC2.
- Set up GitLab CI/CD: linters, tests and automatic DAG deployment after code review.

Python Developer, freelance

Remote

Data collection and processing projects

2023 – 2024 2 yr

WHAT I DID

- Wrote Python scrapers and data-collection scripts: pulled data from websites and APIs, cleaned it in pandas and loaded it into PostgreSQL, Excel and Google Sheets.
- Set up scheduled loads with retries and failure alerts, so exports no longer had to be run by hand.
- Built REST API integrations and Telegram bots for clients' internal work.
- Worked with clients directly: gathered requirements, estimated the work and delivered it.

Stack

Admin → Connections: everything my pipelines talk to every day.

conn_id	type	where and why
Orchestration and queues		
airflow_default	Apache Airflow	38 DAGs, retries, SLAs, failure alerts
celery_workers	Celery	Background tasks, up to 200k a day
redis_cache	Redis	Internal API cache and queues
Storage and search		
postgres_core	PostgreSQL	Schemas for 9 domains, partitioning, indexes
clickhouse_dwh	ClickHouse	Analytical warehouse of 900M+ rows, materialized views
es_search	Elasticsearch	Full-text search across citizen requests
Integrations and processing		
http_external	REST API · JSON · XML	11 external services with schema validation
ws_operators	WebSocket	Real-time stream to the operators' dashboard
python_etl	Python · pandas · SQL	Transformations, quality checks, export automation
Infrastructure		
docker_runtime	Docker Compose · Linux	14 services, environment up in 10 minutes
vault_secrets	HashiCorp Vault	Passwords and access keys outside the code
aws_default	AWS S3 · EC2	Raw data, backups, separate ETL jobs
gitlab_ci	Git · GitLab CI/CD	Linters, tests, automatic DAG deployment

Projects

My own product, plus work outside the job listed as DAGs.

MIRYAS AI

in production

meta-ads-agent-v7dho4qhxa-uc.a.run.app

My own product: an autonomous AI agent that creates and launches Meta ad campaigns on its own. It runs entirely on Google Cloud.

THE DATA ENGINEERING PART

- Collect campaign statistics from the Meta Marketing API on a schedule: Cloud Scheduler triggers the jobs, raw responses land in Cloud Storage, parsed metrics in Cloud SQL.
- Build a mart over campaigns and creatives in BigQuery: spend, impressions, clicks and cost per lead by day.
- Validate the incoming data: gaps in Meta's reports, duplicates by campaign_id, spend that disagrees with the \$50 daily cap.
- The Vertex AI agent reads those same marts and decides which creatives and audiences to keep.

Cloud Run

Cloud SQL

BigQuery

Cloud Storage

Cloud Scheduler

Vertex AI

Meta Marketing API

Python

status	dag_id	schedule	what it does	stack
paused	cbu_rates_pipeline	@daily	Pulls currency rates from the Central Bank of Uzbekistan every day, keeps the history in PostgreSQL and builds a mart in ClickHouse. Checks data quality on every run.	Airflow PostgreSQL ClickHouse Docker
paused	realtime_events_stream	@continuous	Takes events over WebSocket, buffers them in Redis Streams and computes per-minute aggregates in ClickHouse.	WebSocket Redis ClickHouse

status	dag_id	schedule	what it does	stack
paused	vacancy_search	@hourly	Crawls job postings, processes them in Celery queues and serves search over Elasticsearch through a REST API.	Celery Elasticsearch REST API

check_education SQLCheckOperator

queued

 -

Education and languages

Admin → Variables

key	value
education.degree	BSc, Business Information Technology
education.university	Kimyo International University in Tashkent (KIUT)
education.period	2025–2029 studying
languages.uz	native
languages.ru	fluent, C1
languages.en	B2

sink_contacts EmailOperator

queued

 -

Contacts

The DAG's last task: deliver the message.

to
ibroximovdilmurod44@gmail.com

phone +998 88 410 45 45

location Tashkent, Uzbekistan